

Assignment 3

Youssef Amin, Valerie Barker

April 2026

1 Introduction

For this assignment we worked with the sentiment analysis dataset which had a massive feature-set. We implemented a feed forward neural network, tuned hyper-parameters for performance, and implemented k-fold cross validation models and dropout and dropout bagging models.

2 Task 1

Data Preparation

We processed the IMDb dataset by first loading all reviews from the provided directory structure, assigning labels of 1 for positive reviews and 0 for negative reviews. Each review was then cleaned using a preprocessing function that removed HTML tags, converted all text to lowercase, and eliminated non-word characters. Common emoticons were preserved and appended to the cleaned text to retain sentiment information.

After preprocessing, the dataset was shuffled to ensure a random distribution of samples. The data was then split into training and testing sets, using approximately 80% of the data for training and the remaining 20% for testing.

Finally, we transformed the text data into numerical feature vectors using term frequency-inverse document frequency (TF-IDF) representation via `TfidfVectorizer`. The vectorizer was fit on the training data and then applied to both the training and test sets to ensure a consistent feature space. This produced high-dimensional sparse vectors that were used as input to our models.

3 Task 2

FNN Description

The Forward Neural Network was implemented using PyTorch's `nn.Module`. We used a hidden layer structure of 128, 64, 32, 32 with ReLU activation for hidden layers. We tuned these hidden layers to optimize performance.

For training we begin each epoch by clearing gradients, then we calculate our prediction with forward and train with backward and optimizer. For the optimizer we used the Adam model.

4 Task 3

Hyper-Parameters

We found the best performance with the following hyper-parameters:

1. Learning Rate: 0.00140001
2. Epochs: 51
3. Weight Decay: 6.1e-06

Baseline vs Logistic

	Training Accuracy	Test Accuracy	Training Time (s)
Baseline	0.9855	0.9019	178.998
Textbook Logistic Reg.	0.8971	0.8988	304.044

Table 1: Comparison of Baseline and Logistic Regression Performance

The logistic regression implementation from the textbook performed nearly as well as the forward net on the test set but significantly worse on the training set. This suggests the logistic regression model exhibits virtually no overfitting while the forward net exhibits more overfitting. The forward network had a better test accuracy and superior runtime despite possible issues with overfitting.

5 Task 4

K-Fold Implementation

To implement k-fold validation we shuffled and partitioned the training data k times with each partition splitting the training data into a k-1/k and 1/k split for training and validation. Each split chose a different subsection for the validation and let the rest be the training.

After processing the data in this way we trained k models using our sub-training and validation sets and finally evaluated the models by having them predict on the original test set (processed in batches to manage dataset size), averaging their prediction, and using sigmoid activation to make a mean prediction.

We used k=5 and the same hyper-parameters and models for each k fold model as the baseline model to make a fair comparison

Comparison to Baseline

	Training Accuracy	Test Accuracy	Training Time
Baseline	0.9855	0.9019	178.998
K-Fold	0.9869	0.8916	791.559

Table 2: Comparison of Baseline and K-Fold Performance

The baseline model ran significantly faster and had a better performance on the test data. The k-fold cross validation had a very slightly better accuracy on the training set. This suggests the k-fold method may have slightly overfit the data. It's also possible that different hyper-parameters could produce a better result for the k-fold, but we kept them consistent to make an apples to apples comparison.

6 Task 5

Dropping Implementation

For drop-out we used PyTorch's built in dropout feature to select weights to randomly drop.

For bagging we split the training dataset into 5 subsets and trained a dropout model on each subset. Then, we had each model predict on the test set, averaged the results, and used sigmoid to create a final prediction.

We used a dropout rate of 5%

Comparison to Baseline

	Training Accuracy	Test Accuracy	Training Time (s)
Baseline	0.9855	0.9019	178.998
Drop-out Single	0.9752	0.8813	199.799
Drop-out Bagging	0.9370	0.8899	246.342

Table 3: Comparison of Baseline, Drop-out Single, and Drop-out Bagging Performance

The baseline model had a superior performance for training, testing, and training time. Single drop-out has a better training accuracy than bagging, but a slightly worse test accuracy. Bagging took the longest, but significantly reduced the training accuracy, which suggests that it's less overfit. This suggests that you could optimize bagging more by tuning hyper-parameters to produce better results, but we kept the hyper-parameters consistent to make an apples to apples comparison.